

Informe Técnico: Análisis y Estructura del Módulo de Autenticación

Este documento ha sido generado automáticamente utilizando la herramienta **cutImgPDF**. El propósito principal es demostrar la funcionalidad crítica de división y corte horizontal de imágenes extremadamente largas, combinada con la inserción de títulos, subtítulos e imágenes en paralelo.

1. Código del Middleware de Validación de Tokens

El siguiente bloque muestra una captura de pantalla extremadamente vertical (2500 píxeles de alto) que contiene el código completo del middleware. El motor calculará el espacio restante en esta página y recortará la imagen en las partes que sean necesarias para continuar en las páginas siguientes de manera limpia.

```
1      import os
2      import sys
3      from PIL import Image
4
5      # This is a very long code screenshot used to test pdf splitting
6      def process_image_splitting(img_path, page_height):
7          """
8          La lógica matemática calcula la escala y corta horizontalmente.
9          """
10         scale = width / img.width
11         h_scaled = img.height * scale
12         if h_scaled <= page_height:
13             print('Image fits completely')
14             return [img]
15
16         slices = []
17         d = 0.0
18         while d < h_scaled:
19             h_slice = min(page_height, h_scaled - d)
20             y_start = d / scale
21             y_end = (d + h_slice) / scale
22             slice_img = img.crop((0, int(y_start), img.width, int(y_end)))
23             slices.append(slice_img)
24             d += h_slice
25         return slices
26
27     def main_execution_loop():
28         print('Starting automated document assembly')
29         config = load_json('config.json')
30         for item in config:
31             if item['type'] == 'image':
32                 add_image(item['path'])
33             elif item['type'] == 'title':
34                 add_title(item['text'])
35         print('Done generating document')
36
37     import os
38     import sys
39     from PIL import Image
40
41     # This is a very long code screenshot used to test pdf splitting
42     def process_image_splitting(img_path, page_height):
43         """
44         La lógica matemática calcula la escala y corta horizontalmente.
45         """
46         scale = width / img.width
47         h_scaled = img.height * scale
48         if h_scaled <= page_height:
```

```

49     print('Image fits completely')
50     return [img]
51
52     slices = []
53     d = 0.0
54     while d < h_scaled:
55         h_slice = min(page_height - h_scaled - d)
56         y_start = d / scale
57         y_end = (d + h_slice) / scale
58         slice_img = img.crop((0, int(y_start), img.width, int(y_end)))
59         slices.append(slice_img)
60         d += h_slice
61     return slices
62
63     def main_execution_loop():
64         print('Starting automated document assembly')
65         config = load_json('config.json')
66         for item in config:
67             if item['type'] == 'image':
68                 add_image(item['path'])
69             elif item['type'] == 'title':
70                 add_title(item['text'])
71         print('Done generating document')
72
73     import os
74     import sys
75     from PIL import Image
76
77     # This is a very long code screenshot used to test pdf splitting
78     def process_image_splitting(img_path, page_height):
79         """
80         La lógica matemática calcula la escala y corta horizontalmente.
81         """
82         scale = width / img.width
83         h_scaled = img.height * scale
84         if h_scaled <= page_height:
85             print('Image fits completely')
86             return [img]
87
88         slices = []
89         d = 0.0
90         while d < h_scaled:
91             h_slice = min(page_height - h_scaled - d)
92             y_start = d / scale
93             y_end = (d + h_slice) / scale
94             slice_img = img.crop((0, int(y_start), img.width, int(y_end)))
95             slices.append(slice_img)
96             d += h_slice
97         return slices
98
99     def main_execution_loop():
100         print('Starting automated document assembly')
101         config = load_json('config.json')
102         for item in config:
103             if item['type'] == 'image':
104                 add_image(item['path'])
105             elif item['type'] == 'title':
106                 add_title(item['text'])
107         print('Done generating document')
108
109     import os
110     import sys
111     from PIL import Image
112
113     # This is a very long code screenshot used to test pdf splitting
114     def process_image_splitting(img_path, page_height):
115         """

```

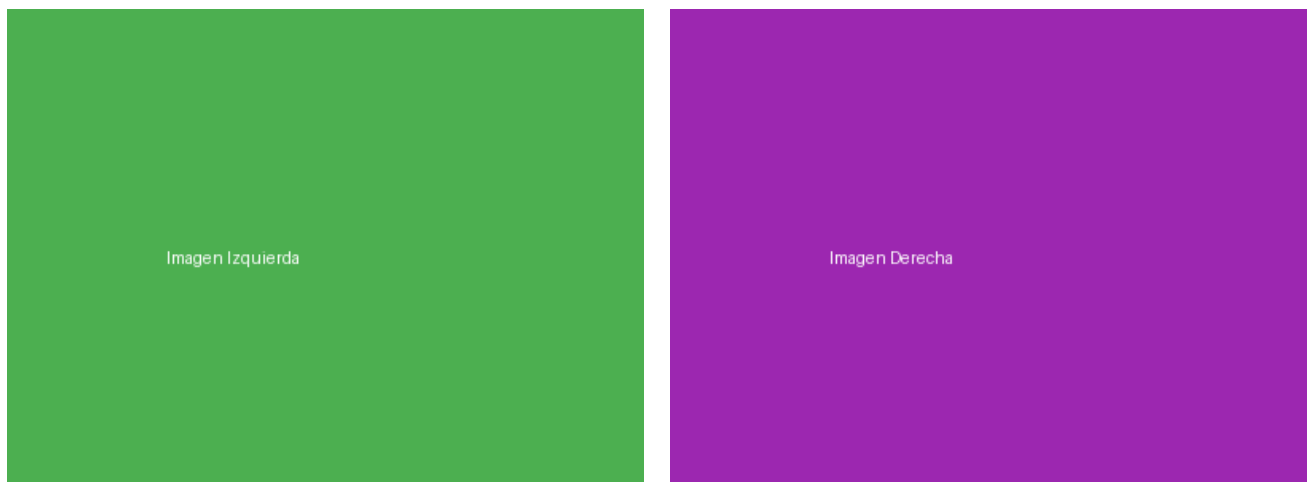
```

116     La lógica matemática calcula la escala y corta horizontalmente.
117     """
118     scale = width / img.width
119     h_scaled = img.height * scale
120     if h_scaled <= page_height:
121         print('Image fits completely')
122         return [img]
123
124     slices = []
125     d = 0.0
126     while d < h_scaled:
127         h_slice = min(page_height, h_scaled - d)
128         y_start = d / scale
129         y_end = (d + h_slice) / scale
130         slice_img = img.crop((0, int(y_start), img.width, int(y_end)))
131         slices.append(slice_img)
132         d += h_slice
133     return slices
134
135 def main_execution_loop():
136     print('Starting automated document assembly')
137     config = load_json('config.json')

```

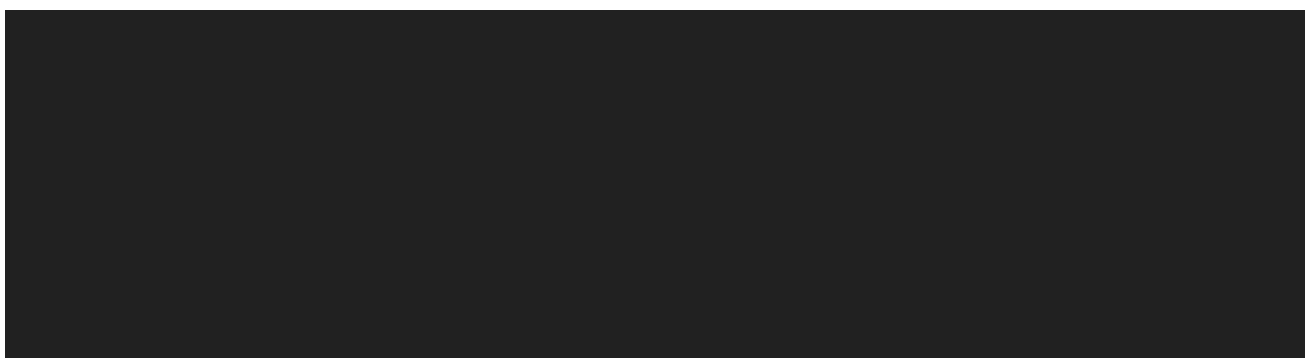
2. Arquitectura y Módulos de Soporte (Comparativa)

Aquí demostramos la inserción de dos imágenes en paralelo (lado a lado). El motor escala cada una para ocupar exactamente el 50% del ancho imprimible disponible y mantiene la alineación horizontal de forma perfecta.



3. Captura del Panel de Control del Servidor

Una captura de pantalla estándar (1200x800 píxeles). El motor la escalará para que ocupe todo el ancho disponible sin deformarla.



Captura de Pantalla Normal